Building an MCP Server for Your Custom Web Components

By Sudeep Parchure — April 2026

What is MCP?

MCP stands for **Model Context Protocol**. It is an open standard, introduced by Anthropic in late 2024, that defines how AI assistants communicate with external tools, data sources, and services.

Think of it like a **USB standard for AI**. Before USB, every device needed a different connector. Before MCP, every AI tool needed a custom integration. MCP fixes that by giving every AI assistant one consistent way to call any tool.

When you configure an MCP server in VS Code or Claude Desktop, the AI can call your tools the same way a developer calls an API — with defined inputs, outputs, and descriptions.

Why MCP Exists

AI assistants are smart, but they are blind to your private systems. Without a structured way to connect them, every team ends up building one-off plugins — fragile, undocumented, and tied to a single AI product.

MCP solves three real problems:

- **Discovery** — the AI can ask "what tools are available?" and get a structured answer
- **Consistency** — one server works with any MCP-compatible client (VS Code, Claude, Cursor, etc.)
- **Portability** — move between AI assistants without rewriting your integrations

In short, MCP turns your internal knowledge — component libraries, APIs, databases — into something an AI can reliably use.

The Problem This Project Solves

Modern frontend teams build **design systems** — shared component libraries with dozens of components, each with their own props, variants, and usage rules.

The pain is real: developers forget component names, mix up prop signatures, and waste time looking things up in Storybook or Figma. Onboarding new team members takes weeks, not days.

> What if your AI coding assistant already knew every component in your library — the props, the snippets, the design tokens — without you having to ask?

That is exactly what this project delivers.

MCP Server vs. Custom AI Agent — Which One to Use?

This is the most common question teams ask when they start exploring AI tooling. The honest answer is: **they solve different problems**.

Use an MCP Server when:

- You have **structured, queryable data** — component manifests, API specs, database records
- You want the tool to work with **any AI assistant**, not just one you build yourself
- You want **fast, lightweight** lookups — not reasoning or decision-making
- You want developers to stay in their IDE and get answers inline
- Setup time matters — MCP servers can be running in under an hour

Use a Custom AI Agent when:

- You need the AI to **take multi-step actions** — e.g. create a ticket, then assign it, then notify Slack
- The task requires **reasoning across multiple sources** that are not structured
- You need a **dedicated chat interface** or a standalone product
- You are building something that must work **without a human in the loop**

The practical rule: if you are augmenting a developer's workflow inside an IDE, use MCP. If you are automating a business process end-to-end, use a custom agent.

For a component library, MCP is the right answer — structured data, instant lookups, works in VS Code today.

The Project: CL Component Library with MCP

GitHub: https://github.com/sudeep31/UIComponentsWithMCP

This is a proof-of-concept built in April 2026. It demonstrates the full journey — from design tokens to a working MCP server — across seven phases.

What Is in the Project

`@cl/tokens` — Design tokens (colours, spacing, typography) generated via Style Dictionary.

`@cl/react` — Six React components: Button, TextBox, NumberBox, Select, TextArea, List. Each is typed, themed via tokens, and documented.

`@cl/web-components` — The same six components compiled as standard Web Components (Custom Elements). Framework-agnostic — works in Angular, Vue, plain HTML, or any future framework.

Storybook — Visual playground for all components. Developers can explore every prop and variant interactively.

Docusaurus — Full documentation site with component API reference, getting-started guide, and design token catalogue.

MCP Server — A Python server (FastMCP) that exposes four AI tools: `list_components`, `get_component_props`, `get_component_snippet`, and `get_design_tokens`.

Architecture

The MCP server is intentionally simple. It reads static JSON manifests and returns structured data. There is no database, no network call, no complex dependency.

```
VS Code / Claude / Cursor
        │
        │  JSON-RPC over stdio
        ▼
  mcp-server/main.py   (FastMCP)
        │
        ├── list_components       → reads manifests/
        ├── get_component_props   → reads manifests/<name>.json
        ├── get_component_snippet → generates ready-to-use code
        └── get_design_tokens     → reads tokens/
```

Each component manifest is a plain JSON file that describes the component — its props, their types, defaults, and whether they are required. The MCP server reads these files and exposes them as AI-callable tools.

Installation

Prerequisites

- Python 3.11 or newer
- Node.js 20 or newer
- pnpm 9 or newer (`npm install -g pnpm`)

Step 1 — Clone the repository

```
git clone https://github.com/sudeep31/UIComponentsWithMCP.git
cd UIComponentsWithMCP
```

Step 2 — Install Node dependencies

```
pnpm install
```

Step 3 — Build all packages

```
pnpm --filter @cl/tokens build
pnpm --filter @cl/react build
pnpm --filter @cl/web-components build
```

Step 4 — Set up the MCP server

```
cd mcp-server
python -m venv .venv
.venv\Scripts\activate        # Windows
pip install -r requirements.txt
```

Step 5 — Verify the server

```
python test_tools.py
```

All 24 tests should pass. The server is now ready.

How the MCP Server Works — The Code

The entire server is about 80 lines of Python. Here is the key logic:

`mcp-server/main.py` — The entry point. FastMCP decorators turn plain Python functions into AI-callable tools.

```python
from fastmcp import FastMCP
import json, pathlib

mcp = FastMCP("CL Component Library")
MANIFESTS = pathlib.Path(__file__).parent / "components" / "manifests"

@mcp.tool()
def list_components() -> list[str]:
    """List all available CL components."""
    return [f.stem for f in MANIFESTS.glob("*.json")]

@mcp.tool()
def get_component_props(component: str) -> dict:
    """Get the props for a specific component."""
    path = MANIFESTS / f"{component}.json"
    return json.loads(path.read_text())

@mcp.tool()
def get_component_snippet(component: str, framework: str = "html") -> str:
    """Get a ready-to-use code snippet for a component."""
    props = get_component_props(component)
    # ... snippet generation logic
    return snippet

if __name__ == "__main__":
    mcp.run()
```

Component manifest — `mcp-server/components/manifests/button.json`:

```json
{
  "name": "cl-button",
  "description": "Primary action button with variant and loading support",
  "props": [
    { "name": "variant", "type": "string", "default": "primary" },
    { "name": "disabled", "type": "boolean", "default": false },
    { "name": "loading", "type": "boolean", "default": false }
  ]
}
```

The pattern is the same for every component. Adding a new component means adding one JSON file — no changes to the server code.

Connecting to VS Code

Once the server is running, register it in your project's `.vscode/mcp.json`:

```
{
  "servers": {
    "cl-components": {
      "type": "stdio",
      "command": "C:/UIComponentsWithMCP/mcp-server/.venv/Scripts/python",
      "args": ["C:/UIComponentsWithMCP/mcp-server/main.py"],
      "cwd": "C:/UIComponentsWithMCP"
    }
  }
}
```

Reload VS Code. Open GitHub Copilot Chat. You can now ask:

- "*What components are available in the CL library?*"
- "*Show me the props for cl-select*"
- "*Give me a snippet for cl-button in Angular*"

The AI answers instantly — from your own library, not from the internet.

Using the Web Components in Any Project

The `@cl/web-components` package compiles every component as a standard Custom Element. It requires no framework.

In plain HTML:

```
<script type="module" src="./dist/custom-elements.js"></script>
<link rel="stylesheet" href="./dist/custom-elements.css" />

<cl-button variant="primary">Save</cl-button>
<cl-textbox label="Email" type="email"></cl-textbox>
```

In Angular:

```
npm link @cl/web-components        # or install from npm
```

```
// main.ts
import "@cl/web-components";

// component.ts
schemas: [CUSTOM_ELEMENTS_SCHEMA];
```

```
<!-- template -->
<cl-button [loading]="isSaving()" (click)="save()">Save</cl-button>
```

The same bundle works in React, Vue, Svelte, or any other framework that supports standard HTML elements.

What I Learned

A few honest observations from building this POC:

MCP servers are simpler than expected. The FastMCP library reduces the server to decorated Python functions. The protocol handles discovery, validation, and transport. You focus on the tools, not the plumbing.

JSON manifests are the right abstraction. Storing component metadata as static JSON means the server starts instantly, needs no database, and is easy to version-control alongside the components themselves.

Web Components solve the framework lock-in problem. Building once in React and exporting as Custom Elements means the same components work in every project — including legacy Angular apps and plain HTML prototypes.

The MCP + Web Components combination is genuinely useful. When Copilot Chat can call `get_component_snippet("cl-button", "angular")` and return accurate code from your actual library, it stops hallucinating component names and prop signatures. That alone saves hours per sprint.

When to Go Further

This POC is a starting point. Here is how to extend it:

- **Add a `search_components` tool** — fuzzy search across all manifest descriptions
- **Connect to Figma** — sync design tokens automatically from your Figma library
- **Publish to npm** — version-tag the packages and install like any other dependency
- **Add a `validate_usage` tool** — let the AI check whether a component is being used correctly in a given file

---

References

- MCP Specification — https://modelcontextprotocol.io
- FastMCP (Python) — https://github.com/jlowin/fastmcp
- Web Components (MDN) — https://developer.mozilla.org/en-US/docs/Web/API/Web_components
- Style Dictionary — https://amzn.github.io/style-dictionary
- Storybook — https://storybook.js.org
- Angular Custom Elements — https://angular.dev/guide/elements
- VS Code MCP Support — https://code.visualstudio.com/docs/copilot/chat/mcp-servers
- **Project on GitHub** — https://github.com/sudeep31/UIComponentsWithMCP

If this was useful, feel free to connect, share, or drop a comment. Happy to answer questions about the implementation.